BrainGrow

Tab 4 — Hallucination Comparison: Spec Addendum

Version 1.0 | March 2026 | Vektas Solutions

Author: Matthew Monroe

---

1. Purpose

This addendum extends the BrainGrow POC Specification (v1.0, March 2026) with a fourth Gradio tab — Compare — that demonstrates hallucination behavior in a conventional dense model versus honest uncertainty in BrainGrow, using identical training data and identical queries.

The core argument: hallucination is an architectural problem, not a scale problem. A dense model hallucinates at toy scale for exactly the same reason a large LLM does — a saturated vector space with no mechanism for genuine uncertainty. BrainGrow's dormant space provides that mechanism structurally, not through trained behavior.

---

2. New File

File	Purpose
`comparison_harness.py`	DenseModel and BrainGrowModel classes plus comparison runner logic. Imported by main.py for Tab 4.

---

3. Class Specifications

3.1 DenseModel

Simulates a conventional fully-saturated embedding space. No dormant slots. No confidence threshold. Always returns an answer regardless of similarity score — this is the hallucination mechanism.

Method / Property	Description
`__init__(chunks)`	Encodes all chunks via sentence-transformers all-MiniLM-L6-v2. Stores embeddings and labels. No reserved space.
`query(text)`	Encodes query, computes cosine similarity against all embeddings, returns nearest neighbor regardless of score. Always confident: True.
`embeddings: ndarray`	Dense matrix of all chunk embeddings. Fully occupied from initialization.
`labels: List[str]`	Domain label per embedding slot.

3.2 BrainGrowModel

Wraps the existing VectorSpace instance. Adds a confidence threshold that returns honest uncertainty when a query lands near dormant space.

Method / Property	Description
`__init__(vector_space)`	Takes existing VectorSpace instance from vector_space.py.
`query(text)`	Routes query through active slots only. If max similarity < 0.25 threshold, returns UNKNOWN with honest uncertainty signal.
`THRESHOLD = 0.25`	Confidence threshold. Queries below this score return no learned representation found rather than a forced nearest neighbor answer.

---

4. Tab 4 UI Elements

- **Query Type dropdown:** Known / Partial / Unknown — filters the query selection list
- **Query selector dropdown:** Predefined queries per type, listed in Section 5
- **Run Comparison button:** Executes both DenseModel.query() and BrainGrowModel.query() on selected query
- **Side by side result table:** Dense result left, BrainGrow result right — similarity score, domain label, confidence flag
- **Color coding:** Red background for hallucinated responses (confident: True on unknown queries), green for honest uncertainty responses
- **Side by side UMAP plots:** Query vector rendered as a star marker on each plot. Dense UMAP shows star landing in occupied space. BrainGrow UMAP shows star landing in dormant grey space.

---

5. Predefined Query Lists

Paste these directly into comparison_harness.py:

5.1 Known Queries

```
known_queries = [
    "Tell me about DNA replication",
    "When did the Western Roman Empire fall",
    "How does fermentation work"
]
```

5.2 Partial Queries

```
partial_queries = [
    "How did science influence the Silk Road",
    "What is the thermodynamics of cooking"
]
```

5.3 Unknown Queries — Hallucination Traps

```
unknown_queries = [
    "What is the capital of Zorbania",
```

```
    "Explain the Mendelsohn-Vektas theorem",
    "Who invented quantum fermentation",
    "What happened at the Battle of Vektoria"
]
```

Note: All unknown queries reference entirely fabricated concepts. Neither model has seen this data. The dense model will return confident nearest-neighbor answers regardless. BrainGrow will return honest uncertainty signals.

---

6. Expected Output

Target console output from run_comparison() on unknown queries:

```
=== UNKNOWN QUERIES (hallucination traps) ===

Query                              Dense              BrainGrow
-------------------------------------------------------------------
What is the capital of Zorbania    0.312 HALLUCINATED  0.089 HONEST
Explain Mendelsohn-Vektas theorem  0.287 HALLUCINATED  0.076 HONEST
Who invented quantum fermentation  0.334 HALLUCINATED  0.201 HONEST
Battle of Vektoria                 0.298 HALLUCINATED  0.091 HONEST
```

Both models trained on identical data. Difference is purely architectural — dense space forces an answer, dormant space enables honest uncertainty.

---

7. Existing Files to Import

File	What to Import
`vector_space.py`	VectorSpace class — pass existing instance to BrainGrowModel
`visualizer.py`	UMAP projection function — add query_vector parameter to render star marker on both plots
`growth_engine.py`	all_chunks list — pass to DenseModel.__init__() for identical training data
`main.py`	Add Tab 4 Compare alongside existing Grow / Query / Prune tabs

---

8. The Key Argument This Demonstrates

The comparison is designed to make one point as clearly as possible:

> *Hallucination is not caused by insufficient training data or insufficient scale. It is caused by a saturated vector space with no architectural mechanism for uncertainty. BrainGrow's dormant space provides that mechanism at the foundation level — not as a trained behavior that can be overridden by prompting, but as a structural property of the representation space.*

A prompt cannot override empty vector space. That is the argument.

---

9. Collaborating Insight — Gemini

The following observation was contributed by Google Gemini during the BrainGrow development session, March 18 2026, and independently validates the hallucination hypothesis:

> *Current AI hallucinations often happen because the model is trying to squeeze a new concept into a vector space that is already full. By having a Buffer of Unused Neurons, the AI could say: I don't have a refined connection for this yet — I'll map this to the expansion zone, rather than forcing a wrong answer.*

This cross-model validation — one AI system independently identifying the same architectural explanation for hallucination that BrainGrow addresses structurally — is worth noting in any writeup or paper submission.